Coronary Risk Assessment

Multimodal AI Hackathon 2026, Track A: Cardiovascular Risk Visualization & Prediction

Team: Heartly (Akinsoyinu Anjola, Mukadam Jamaldeen)

Repo: https://github.com/Anjolaanuoluwapo101/cad-predict

Demo video: https://youtu.be/IuQxSiEEJPc

Overview

This project is an interactive web-based system for the prediction and visualization of coronary artery disease (CAD) risk. From a patient's **demographic, clinical, ECG, laboratory and echocardiographic data**, it predicts overall CAD status and the presence of significant stenosis (50% narrowing or more) in the three principal coronary vessels: the left anterior descending (LAD), left circumflex (LCX) and right coronary artery (RCA). The predictions are mapped onto an interactive 3D anatomical heart, in which each vessel is coloured according to its predicted risk. A clinical dashboard displayed alongside the 3D view presents the overall and vessel-specific probabilities and uses SHAP to explain which patient factors contributed to each prediction.

NOTE: Each vessel output is the model's estimated probability that the vessel has stenosis, that's, 50% or more narrowing of at least one vessel (the dataset's definition of a stenotic vessel); **it is a measure of model confidence, not of how severe the narrowing is.**

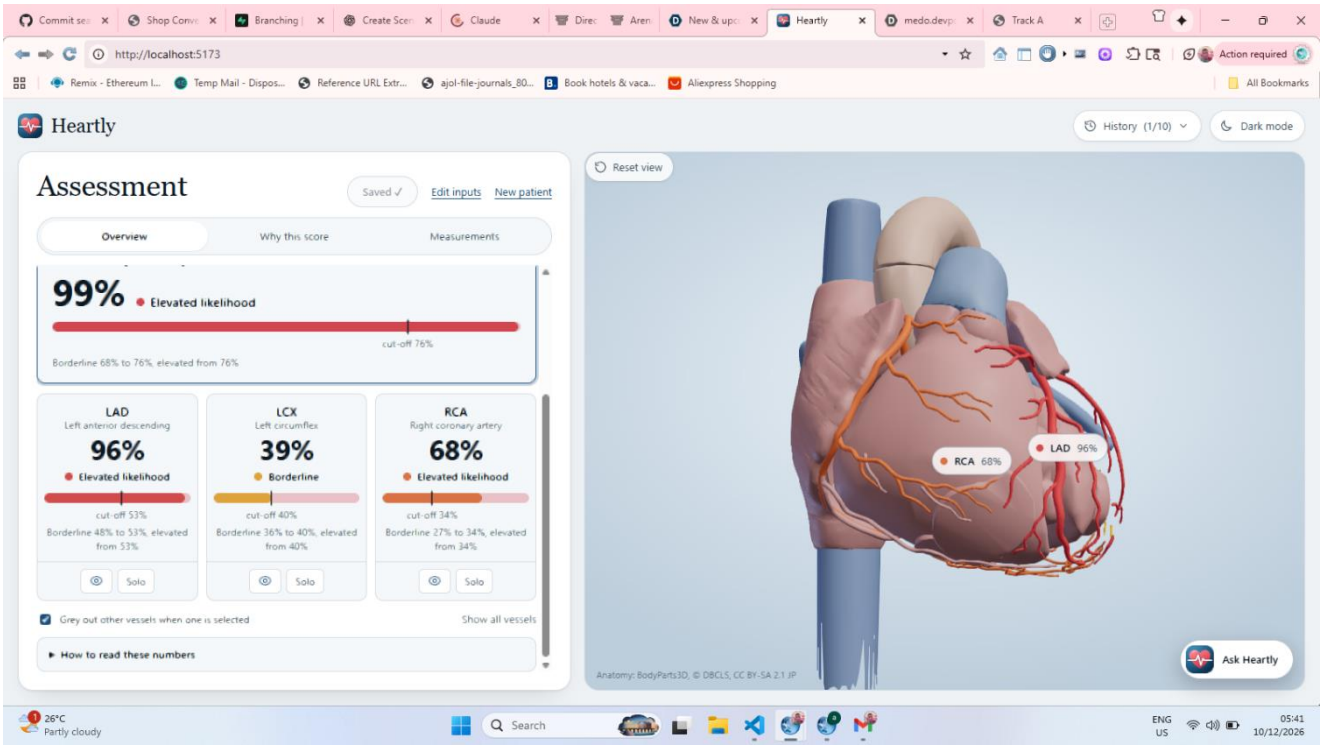


Figure 1. The finished app. Left: overall CAD plus the LAD, LCX and RCA cards, each read against its own cut-off. Right: the 3D heart, coloured by the same scores.

NOTE:

99% CAD means the model is 99% sure that one of the three target coronary vessels is stenotic.

96% LAD means that the model is 96% sure that, the LAD artery is at least narrowed by 50% or more.

39% LCX means that the model is 39% sure that, the LCX artery is at least narrowed by 50% or more.

1. Dataset Pre-processing

The data comes from the Extension of Z-Alizadeh Sani dataset (UCI repository): 303 patients and 59 columns, with no missing cells and no duplicate rows. Each patient has **demographic, history, examination, ECG, echocardiographic and laboratory values**, plus four labels: overall CAD (the Cath column) and stenosis in the LAD, LCX and RCA. In total, 216 patients are labelled CAD and 87 normal. By vessel, 177 patients are labelled stenotic in the LAD, 119 in the LCX and 114 in the RCA.

Before any modelling, the file was audited:

- **Target leakage.** The four label columns (LAD, LCX, RCA and Cath) are excluded from the inputs, as the brief requires, because they hold the answers. This exclusion is enforced in code and covered by an automated test.
- **Constant column.** Exertional CP is "N" for every patient, so it carries no information and was dropped.
- **Inconsistent label.** One patient's overall label (Excel row 95) contradicts that patient's three vessel labels. We removed this patient from the CAD model only (302 patients) and kept that particular row for the vessel models only.
- **Encoding.** Yes/No columns became 1/0. Sex became a Male flag, with 1 confirming it is a Male and 0 if it is a female. Valve disease became an ordinal scale from 0 to 3, and bundle branch block became two flags (LBBB, RBBB). The result is **55 input features**.
- **Derived values.** BMI and the Obesity flag are calculated from weight and height, so the form does not ask for them. The Obesity column means "BMI of about 25 or higher" rather than clinical obesity, so we call it "BMI over 25".

2. Model architecture

The system works like a production line with six stations: clean the data, test three candidate models, pick the best, make its percentages trustworthy, explain each result, and save the final models for the app.

Testing the candidates: Four questions were asked of every patient: CAD, LAD, LCX and RCA. Each has its own model, and all four read the same patient information. Three kinds of model compete on each question, all on default settings: **logistic regression**, a **random forest** and **gradient boosting**. A "do-nothing" model that always guesses the most common answer sets the floor.

Picking the best model: To pick the best model for each question, the patients are split into 5 equal groups. Each group takes a turn as the hidden group: the model trains on the other four, then is tested on the one it has never seen. The whole process is repeated 5 times with a fresh shuffle, giving **25 tests** (this is called cross-validation). Every model faces the same 25 tests, which adds up to 400 runs. Scaling and gap-filling are learned from the training patients only, so test patients never leak in.

Each model is scored by **ROC-AUC**. Pick one sick and one healthy patient at random, and the score is how often the model rates the sick one higher (0.5 is a coin flip, 1.0 is perfect). The best average over the 25 tests wins (Table 1). Scores vary by about 0.03 to 0.06 between tests, so gaps of 0.01 or less count as ties.

Target	Logistic reg.	Random forest	Grad. boosting	Chosen model	Gap to runner-up
CAD	0.929	0.936	0.922	**Random forest**	0.008
LAD	0.826	0.851	0.814	**Random forest**	0.025
LCX	0.687	0.728	0.725	**Random forest**	0.003 (a tie)
RCA	0.722	0.712	0.696	**Logistic reg.**	0.010

Table 1. Mean cross-validated ROC-AUC over 25 rounds. The standard deviation across rounds is about 0.03 to 0.06, so gaps of 0.01 or less are effectively ties.

Leakage check: As a safety test, the answers were shuffled at random and every model was run again. A model secretly peeking at the answers would still score well. Instead, all of them dropped to coin-flip level (0.41 to 0.58), while the real models scored between 0.69 and 0.94. The real scores therefore come from real patterns in the data.

Trustworthy percentages: A model's raw score is not always a true percentage. Once the **best model is picked**, its scores are corrected with **Platt scaling**, a small adjustment learned from the model's predictions on the hidden patients in the 25 tests. After this, "70%" really means that about 70 in 100 similar patients have the condition (Figure 6). A **Cut-off** is the score at which the app switches to "Elevated likelihood". It is set where the model best separates sick patients from healthy ones, and each question has its own: CAD 0.76, LAD 0.53, LCX 0.40 and RCA 0.34. Every score is therefore read against its own cut-off. A CAD score of 39% is far below 76%, so it is low. An LCX score of 39% is almost at 40%, so it needs attention.

Borderline zone: Borderline is a warning zone just below the cut-off, where the model is close to saying "Elevated". It starts at 90% of the cut-off (80% for RCA). The zone exists because blank optional fields are filled with typical dataset values, which can hide real risk. A score in this zone shows as Borderline instead of Lower, and the app lists which inputs were estimated.

Explaining results: SHAP measures how much each patient detail pushed a score up or down. The dashboard shows each detail's share of the explanation, so the biggest drivers are easy to spot.

Final models and form: The winning models are retrained on all patients and saved with their calibration and cut-off. The form has 52 fields, of which 34 are required: the details that mattered most to each model, plus Height and ten more that made a short form agree better with the full one. **A test then compared the 34-field form with the full form (52) for all 303 patients. The yes/no answer changed for 1 patient on CAD, none on LAD and LCX, and 9 on RCA (3.0%). These were the same patients the models learned from, so this is a sanity check, not proof.**

Finally, the architecture is built to grow. `schema.py` defines every field once, the form is generated from the `/features` endpoint, and the backend only wires HTTP to the predictor, so adding a feature, a model or a vessel does not mean a redesign.

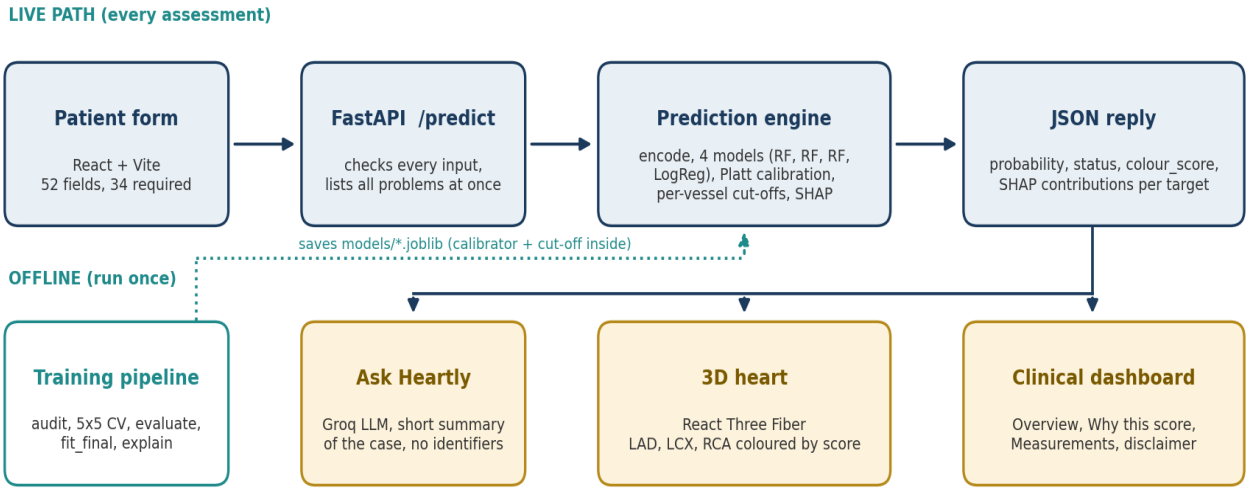


Figure 2. System architecture. The training pipeline runs once and saves the models; everything else happens on every assessment.

3. 3D Pipeline Setup

The heart model comes from **BodyParts3D** (Database Center for Life Science), chosen because it is open, anatomically detailed and licensed for reuse. It arrives as hundreds of tiny separate 3D pieces, so a small script (scripts/build-heart.mjs) was written to assemble them. The script reads the database's "part-of" structure, groups the pieces into eight named parts (LAD, LCX, RCA, LeftMain, Heart_body, Aorta, Pulmonary_trunk and Veins), smooths the joins and exports a single 2.6 MB 3D file (.glb). The LAD has 9,444 triangles, the LCX 3,018 and the RCA 14,682, with about 109,000 in total. The model is kept deliberately light so that it stays responsive without a dedicated graphics card.

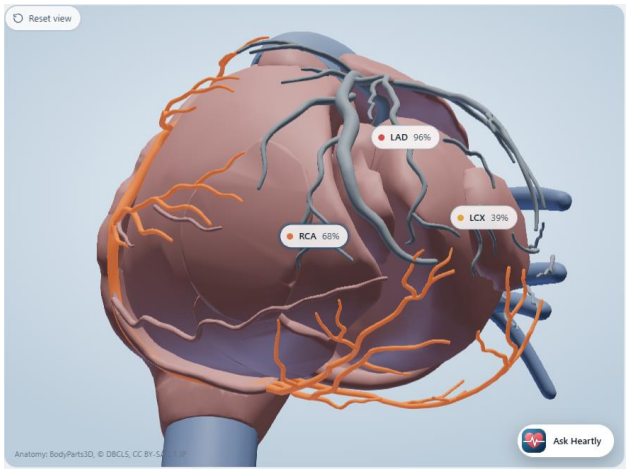


Figure 3. RCA selected: the camera moves to it, the others grey out, and every vessel keeps its percentage label.

The viewer is React with Three.js through React Three Fiber and drei. Because LAD, LCX and RCA are separate named meshes, each one is matched one-to-one with the same key the API returns. That is how the picture and the prediction can never drift apart.

From score to colour: Each vessel gets a `colour_score` from the API, scaled so that amber sits exactly at that vessel's own cut-off. Teal means lower, amber means near the cut-off and crimson red means elevated. **Red-versus-green so the scale works for colour-blind users**, and every vessel also carries a text label and a percentage, so colour is never the only clue.

Users can rotate and zoom the heart, click any vessel to select it (the others grey out), use the eye and Solo toggles on each card, and press Reset view. The target chips in the dashboard follow the 3D selection. Left-Main stays neutral grey, because none of our models predicts it. And one honest note: this is illustrative anatomy, not a scan of the patient. Only the colours change.

4. Usage Instructions

Setup (Windows PowerShell, from the project folder). The saved models must be loaded with the same library versions that trained them (scikit-learn 1.8.0, numpy 2.4.4), which `requirements.txt` pins.

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Run the backend (API docs appear at `/docs`):

```
uvicorn api.main:app --reload
```

Run the app in a second window, then open http://localhost:5173:

```
cd frontend
npm install
npm run dev
```

Optional. To retrain, run `python -m src.train_cv`, then `src.evaluate`, `src.fit_final` and `src.explain`. Trained models are already in `models/`. Run the tests with `python -m pytest -q`. For the Heartly assistant, put a free Groq key in `frontend/.env.local` as `VITE_GROQ_API_KEY` and restart `npm run dev`.

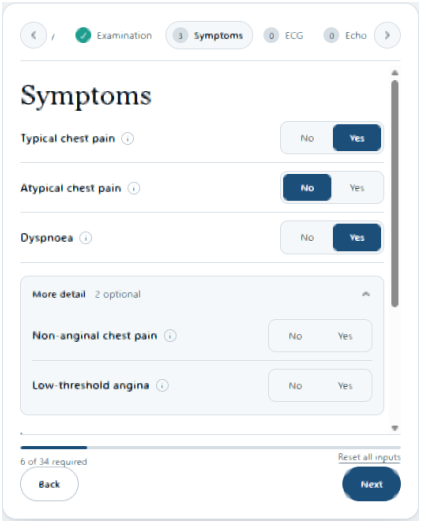


Figure 4. The guided form: seven steps, tooltips, yes/no switches and a progress bar for the 34 required fields.

Using it, in five steps.

1. **Fill in the form.** Seven short steps (Demographics, History, Examination, Symptoms, ECG, Echo, Laboratory). Required fields start empty on purpose, so nothing is submitted by accident. Sliders show typical adult ranges, lab values can switch between mg/dL and mmol/L, and optional fields hide under "More detail".
2. **Press Run.** Missing required fields turn red and the form jumps to the first gap.
3. **Read the Overview.** CAD and the three vessel cards show the likelihood, the status (Lower likelihood, Borderline or Elevated likelihood) and the cut-off. The same colours appear on the heart.
4. **Ask why.** "Why this score" shows the SHAP bars for each target, and "Measurements" shows the patient's own values with how much each one pushed the score up or down.
5. **Ask Heartly (optional).** A chat assistant, powered by Groq, explains the result in plain words. It only receives a short summary with no name or ID, and it is told not to diagnose or advise treatment.

The last ten assessments are saved in the browser and can be restored with one click, there is a dark mode, and the safety disclaimer stays pinned at the bottom of the dashboard so it never scrolls away.

5. Evaluation results

All results come from the 25 cross-validation tests described in Section 2. For the final numbers, the calibration and cut-off were set using the training patients only. The model was then scored on the hidden patients, who played no part in those choices, so the results are not inflated. Table 2 shows every metric the brief requires, at each target's own cut-off.

Target	Accuracy	Precision	Recall	F1	ROC-AUC	Specificity
CAD	0.851	0.962	0.824	0.888	**0.930**	0.919
LAD	0.782	0.801	0.835	0.817	**0.843**	0.708
LCX	0.683	0.584	0.691	0.631	**0.727**	0.678
RCA	0.659	0.538	0.702	0.607	**0.718**	0.634

Table 2. Honest (nested) cross-validated results at each target's own cut-off. Recall is the same thing as sensitivity.

Table 2 gives several measures, because no single number tells the full story. Accuracy is the share of patients classified correctly, but it can mislead. About 71% of patients in the data have CAD, so answering "CAD" for everyone would already score 71%. ROC-AUC is therefore the main measure. It answers one question: if one sick and one healthy patient are picked at random, how often does the model give the sick patient the higher risk score? A score of 0.5 is a coin flip and 1.0 is perfect. CAD scores 0.93, so the model ranks the sick patient higher in about 93 of 100 such pairs. LAD scores 0.84. LCX and RCA score about 0.72, so their results deserve more caution.

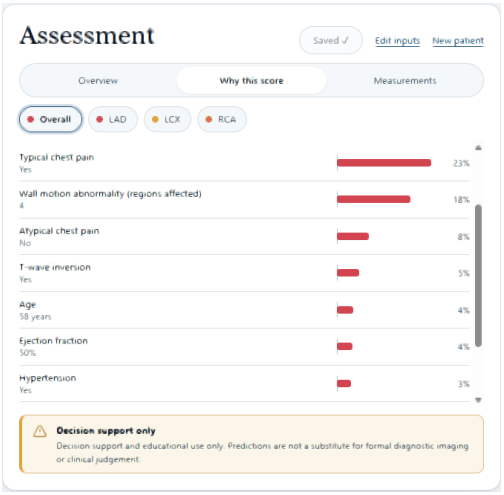


Figure 5. "Why this score": each bar is that factor's share of the explanation.

What drives the predictions. SHAP shows which patient details moved each score the most. For CAD and LAD, these are chest pain, wall motion abnormality, age and ejection fraction. For LCX, age leads, and for RCA, diabetes, sex and age. Figure 5 shows this for the test patient.

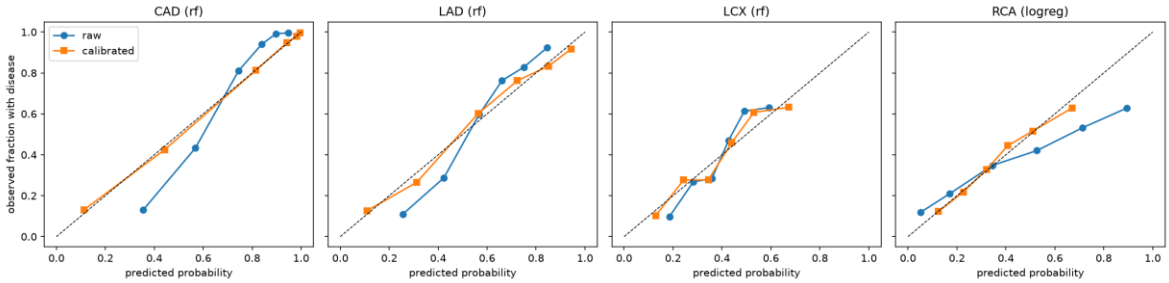


Figure 6. Calibration before (blue) and after (orange) Platt scaling. Orange hugging the dashed diagonal means "70%" really does mean about 70%.

6. Limitations, safety and credits

Limitations, said plainly.

- The data is 303 patients from one centre with no external validation, and the LCX and RCA models are weak, so their cut-offs are approximate.
- The final models were trained on all 303 patients, so scores for those same patients look too confident. The performance claims come only from cross-validation.
- SHAP shows associations in this dataset, not causes. Tooltips and typical ranges are general adult guidance and have not been reviewed by a clinician.

- The 3D vessels are fixed shapes (only colour changes), and Heartly sends a short anonymous case summary to a third party (Groq), so no real patient identifiers should be typed.

Safety. A visible disclaimer is pinned to the dashboard at all times: predictions are for decision support and education only, and are not a substitute for formal diagnostic imaging.

Credits and licences. Anatomy: BodyParts3D, © The Database Center for Life Science, licensed under CC Attribution-Share Alike 2.1 Japan (credit shown on the 3D view; the share-alike condition applies to the model file, not our code). Data: UCI Extension of Z-Alizadeh Sani dataset. Built with scikit-learn, SHAP, FastAPI, React, Three.js and React Three Fiber.

References. Alizadehsani R. et al. (2013) A data mining approach for diagnosis of coronary artery disease. *Computer Methods and Programs in Biomedicine* 111(1). Alizadehsani R. et al. (2016) Coronary artery disease detection using computational intelligence methods. *Knowledge-Based Systems* 109. Mitsuhashi N. et al. (2009) BodyParts3D: 3D structure database for anatomical concepts. *Nucleic Acids Research* 37.